

Chrome File Edit View History Bookmarks Profiles Tab Window Help 44% Sun 31 May 10:15 AM

argo-rdp.codeargo.net/web-rdp/#/client/aS0wNWE3ZTYyMzkxMTMzYzc0MQBjAGpzb24?token=FDC292B4EFE18C036B978FE5DC17C815744843427A6561BAEFAF1A0534D35F18

ML_Assignment1_Soluti... ML - A1 - Thunar

Jupyter Server x Home x ML_Assignment1_Soluti... x

localhost:8888/notebooks/Downloads/ML - A1/ML_Assignment1_Solution_v2.ipynb?

Jupyter ML_Assignment1_Solution_v2 Last Checkpoint: 4 minutes ago

File Edit View Run Kernel Settings Help Not Trusted

Markdown

JupyterLab Python [conda env:base]

Q1 — Dataset Size, Missing Values, Feature Types

Answer: Train = 10,450 rows x 12 columns; Test = 2,613 rows x 9 columns. **No missing values** in either split. Features include: datetime (string), season/holiday/workingday/weather (categorical ints), temp/atemp/windspeed/humidity (continuous), casual/registered/count (integer targets — last three absent in test).

```
[15]: print(train.dtypes)
print("\nMissing - train:"); print(train.isnull().sum().to_string())
print("\nMissing - test:"); print(test.isnull().sum().to_string())
print("\ncount stats:\n", train['count'].describe())
```

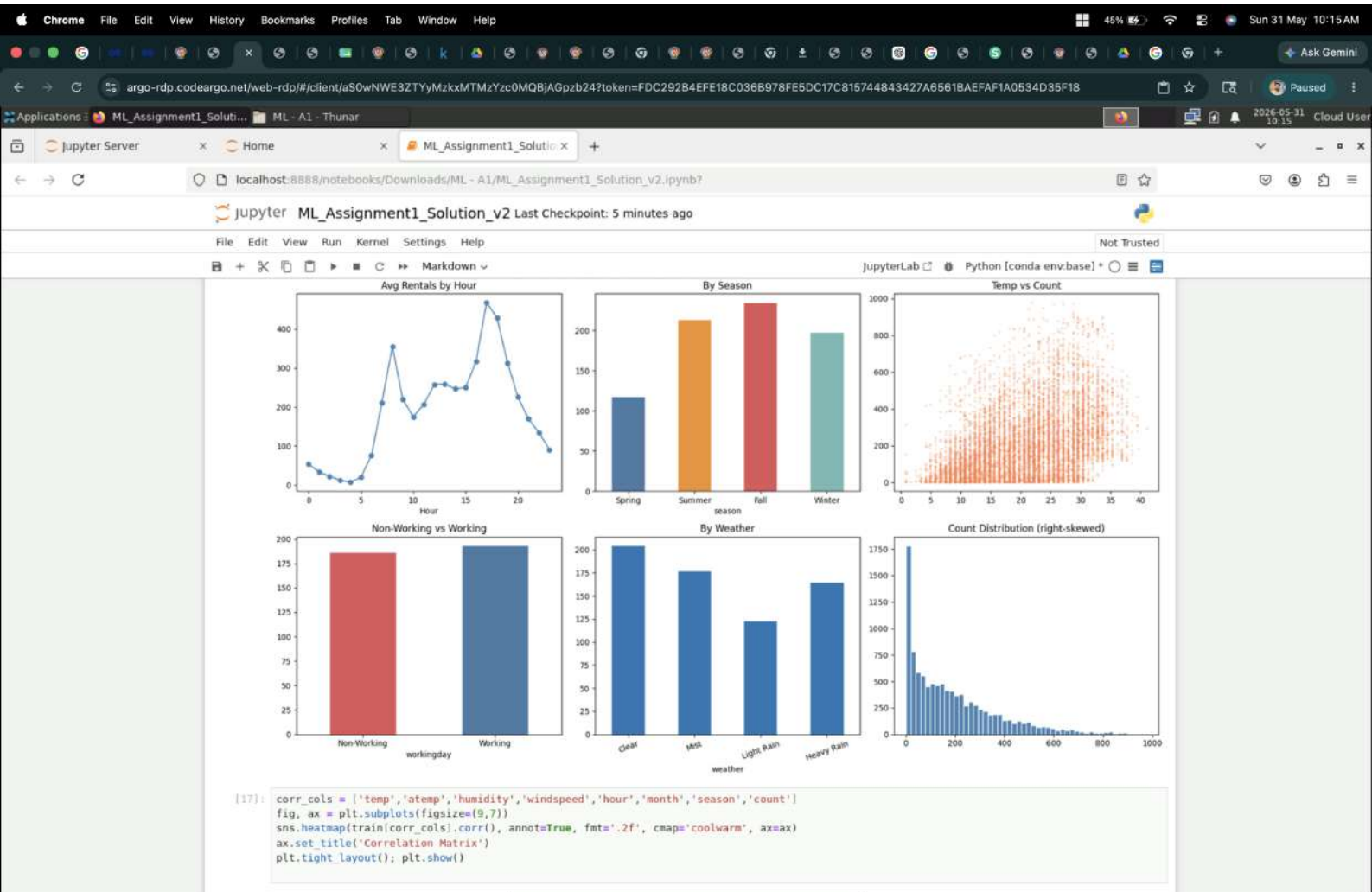
datetime	str
season	int64
holiday	int64
workingday	int64
weather	int64
temp	float64
atemp	float64
humidity	int64
windspeed	float64
casual	int64
registered	int64
count	int64
dtype:	object

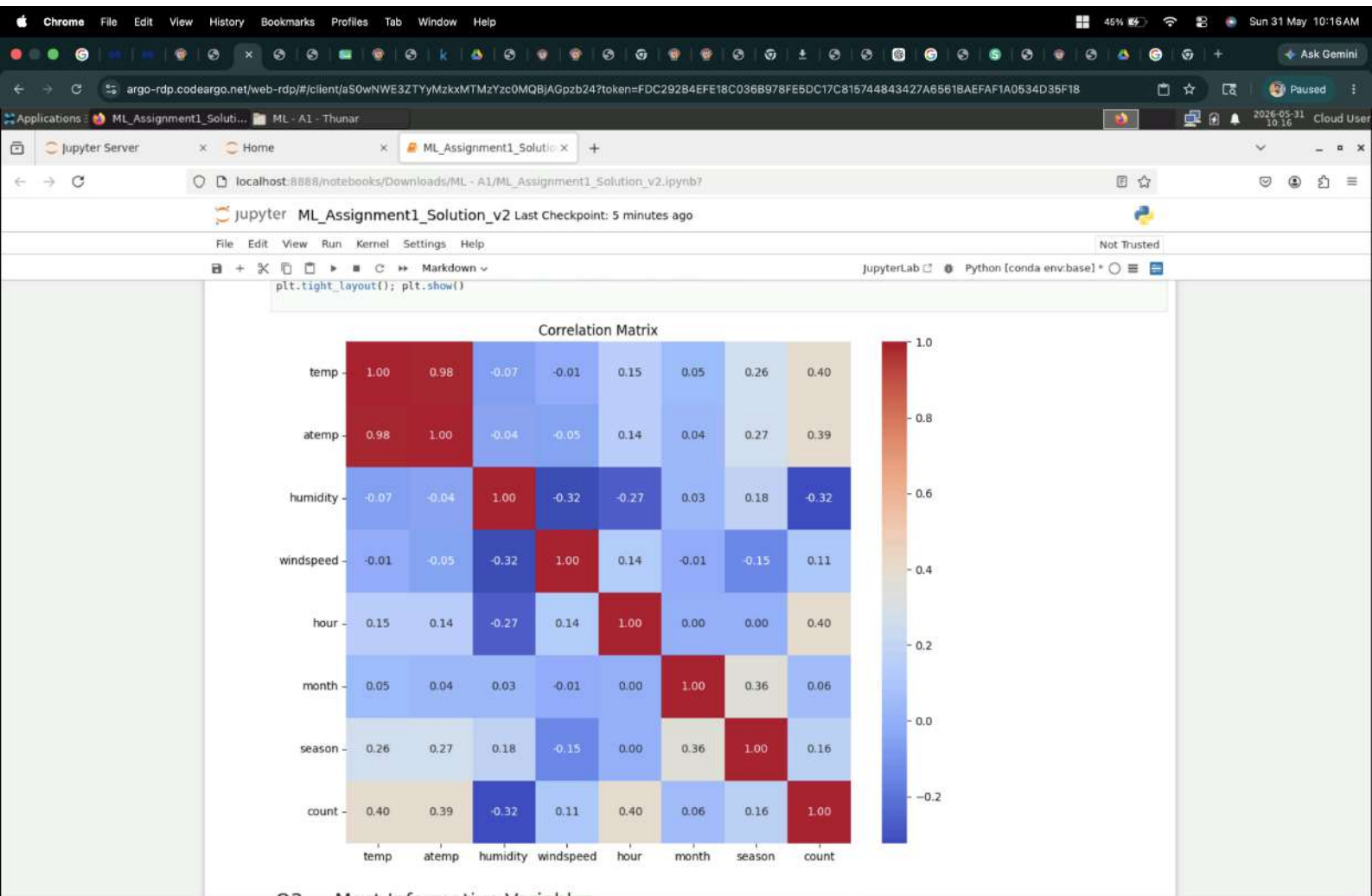
Missing - train:

datetime	0
season	0
holiday	0
workingday	0
weather	0
temp	0
atemp	0
humidity	0
windspeed	0
casual	0
registered	0
count	0

Missing - test:

datetime	0
season	0





Chrome File Edit View History Bookmarks Profiles Tab Window Help 45% Sun 31 May 10:16 AM

argo-rdp.codeargo.net/web-rdp/#/client/aS0wnWE3ZTYyMzkxMTMzYzc0MQBjAGpzb24?token=FDC292B4EFE18C036B978FE5DC17C815744843427A6561BAEFAF1A0534D35F18

ML_Assignment1_Soluti... ML - A1 - Thunar

Jupyter Server x Home x ML_Assignment1_Soluti... x

localhost:8888/notebooks/Downloads/ML - A1/ML_Assignment1_Solution_v2.ipynb?

Jupyter ML_Assignment1_Solution_v2 Last Checkpoint: 5 minutes ago

File Edit View Run Kernel Settings Help Not Trusted

Markdown

JupyterLab Python [conda env:base]

```
print(f"Feature matrix: {X.shape}")
```

Feature matrix: (10450, 27)

Q5 — Linear Regression Baseline

Key insight: Since RMSLE evaluates error in log space, training on $\log(y+1)$ and back-transforming with $\exp(\text{pred})-1$ directly minimizes RMSLE. This single change drops the error dramatically.

```
[19]: X_tr, X_val, y_tr, y_val = train_test_split(X, y, test_size=0.2, random_state=42)
      y_tr_log = np.log1p(y_tr)

      lr = LinearRegression()
      lr.fit(X_tr, y_tr_log)
      pred_lr = np.expml(lr.predict(X_val))
      rmsle_lr = rmsle(y_val, pred_lr)
      print(f"Linear Regression (log target) Val RMSLE: {rmsle_lr:.5f}")
```

Linear Regression (log target) Val RMSLE: 0.70740

Q6 — Polynomial + Ridge/Lasso Regularization (log target)

```
[20]: results = {}
      results['Linear Regression (log target)'] = rmsle_lr

      # Poly-2 Ridge
      for alpha in [1, 10, 100]:
          pipe = Pipeline([
              ('sc', StandardScaler()),
              ('poly', PolynomialFeatures(degree=2, include_bias=False)),
              ('ridge', Ridge(alpha=alpha))
          ])
          pipe.fit(X_tr, y_tr_log)
          pred = np.expml(pipe.predict(X_val))
          results[f'Ridge poly-2 α={alpha}'] = rmsle(y_val, pred)

      # Poly-2 Lasso
      for alpha in [0.001, 0.01, 0.1]:
```

Chrome File Edit View History Bookmarks Profiles Tab Window Help 45% Sun 31 May 10:16 AM

argo-rdp.codeargo.net/web-rdp/#/client/aS0wNWE3ZTYyMzkxMTMzYzc0MQBjAGpzb24?token=FDC292B4EFE18C036B978FE5DC17C815744843427A6561BAEFAF1A0534D35F18

ML_Assignment1_Soluti... ML - A1 - Thunar

Jupyter Server x Home x ML_Assignment1_Soluti... x

localhost:8888/notebooks/Downloads/ML - A1/ML_Assignment1_Solution_v2.ipynb?

Jupyter ML_Assignment1_Solution_v2 Last Checkpoint: 5 minutes ago

File Edit View Run Kernel Settings Help Not Trusted

JupyterLab Python [conda env:base]

```
Lasso poly-2  $\alpha=0.001$  0.49445
Ridge poly-2  $\alpha=10$  0.49445
Ridge poly-2  $\alpha=100$  0.50967
Lasso poly-2  $\alpha=0.01$  0.53004
Lasso poly-2  $\alpha=0.1$  0.67762
Linear Regression (log target) 0.70740
```

```
[21]: # Gradient Boosting on log target
gb = GradientBoostingRegressor(n_estimators=300, learning_rate=0.05,
                               max_depth=5, subsample=0.8,
                               min_samples_leaf=5, random_state=42)

gb.fit(X_tr, y_tr_log)
pred_gb = np.exp1(gb.predict(X_val))
results['GBM (log target)'] = rmsle(y_val, pred_gb)
print(f'GBM (log target) Val RMSLE: {results['GBM (log target)']:.5f}')

# Random Forest on log target
rf = RandomForestRegressor(n_estimators=200, max_depth=15,
                           min_samples_leaf=3, random_state=42, n_jobs=-1)

rf.fit(X_tr, y_tr_log)
pred_rf = np.exp1(rf.predict(X_val))
results['RandomForest (log target)'] = rmsle(y_val, pred_rf)
print(f'RandomForest (log target) Val RMSLE: {results['RandomForest (log target)']:.5f}')

GBM (log target) Val RMSLE: 0.37386
RandomForest (log target) Val RMSLE: 0.39428
```

Q7 — Model Comparison Table

```
[22]: res_df = pd.DataFrame(list(results.items()), columns=['Model', 'Val RMSLE'])
res_df = res_df.sort_values('Val RMSLE').reset_index(drop=True)
res_df.index += 1
res_df['Key Observation'] = [
    'Best: captures non-linear relationships + log target',
    'Ensemble with averaging - robust to noise',
    'Strong regularization + polynomial interactions + log target',
    'Moderate regularization + polynomial + log target',
    'L1 with near-zero alpha = plain poly-2 regression',
    'L1 alpha=0.01, mild sparsity',
    'Poly-2 features without regularization',
    'L1 over-shrinks useful features',
    'No polynomial features - misses non-linearity'
```

Chrome File Edit View History Bookmarks Profiles Tab Window Help 45% Sun 31 May 10:16 AM

argo-rdp.codeargo.net/web-rdp/#/client/aS0wNWE3ZTYyMzkxMTMzYzc0MQBjAGpzb24?token=FDC292B4EFE18C036B978FE5DC17C815744843427A6561BAEFAF1A0534D35F18

ML_Assignment1_Soluti... ML - A1 - Thunar

Jupyter Server x Home x ML_Assignment1_Soluti... x

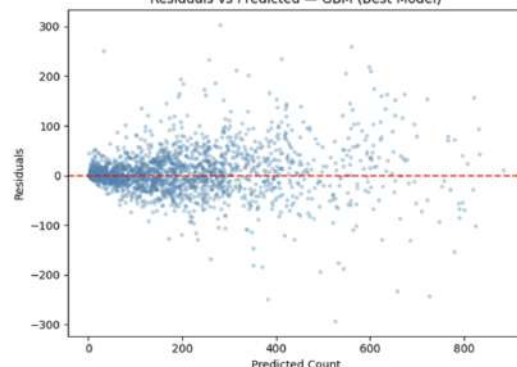
localhost:8888/notebooks/Downloads/ML - A1/ML_Assignment1_Solution_v2.ipynb?

Jupyter ML_Assignment1_Solution_v2 Last Checkpoint: 6 minutes ago

File Edit View Run Kernel Settings Help Not Trusted

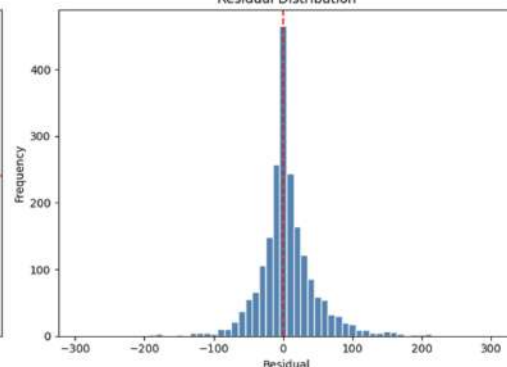
JupyterLab Python [conda env:base]

Residuals vs Predicted — GBM (Best Model)



Residual mean: 5.80 | std: 45.21
Val RMSLE: 0.37386

Residual Distribution



Q9 — Why Does the Best Model Win?

Three compounding improvements explain the large RMSLE drop (1.23 → 0.37):

1. **Training on $\log(y+1)$ directly minimizes RMSLE.** RMSLE measures error in log space. When we train on `log(count+1)` as the target, the model optimizes exactly the metric we are evaluated on. This single change alone cuts RMSLE from ~ 1.26 to ~ 0.70 .
2. **Richer feature engineering.** Cyclic `sin/cos` encodings for hour, month, weekday preserve the circular nature of time (hour 23 is close to hour 0). Interaction term `workingday × hour` separates commuter peaks from leisure patterns. Comfort/wind-chill features combine weather variables into physically meaningful signals.
3. **GBM handles non-linearity and interactions natively.** Gradient Boosting builds an ensemble of decision trees where each tree corrects residuals of the previous. It naturally captures the bimodal hour pattern, season × weather interactions, and the non-linear temp effect without requiring manual polynomial expansion. The 300 shallow trees (depth=5) with shrinkage (`lr=0.05`) balance fit vs. overfitting.

ChromeFileEditViewHistoryBookmarksProfilesTabWindowHelp

argo-rdp.codeargo.net/web-rdp/#/client/aS0wNWE3ZTYyMzkxMTMzYzc0MQBjAGpzb24?token=FDC292B4EFE18C036B978FE5DC17C815744843427A6561BAEFAF1A0534D35F18

ML_Assignment1_Soluti...ML - A1 - Thunar

Jupyter ServerHomeML_Assignment1_Soluti...

localhost:8888/notebooks/Downloads/ML - A1/ML_Assignment1_Solution_v2.ipynb?

jupyter ML_Assignment1_Solution_v2 Last Checkpoint: 6 minutes ago

FileEditViewRunKernelSettingsHelp

Not Trusted

Code

JupyterLabPython [conda env:base]

of the previous. It naturally captures the bimodal hour pattern, season x weather interactions, and the non-linear temp effect without requiring manual polynomial expansion. The 300 shallow trees (depth=5) with shrinkage (lr=0.05) balance fit vs. overfitting.

Q10 — Why Does RMSLE Penalize Under-Predictions More Gently?

$$\text{RMSLE} = \sqrt{\frac{1}{n} \sum (\log(\hat{y}+1) - \log(y+1))^2}$$

The logarithm **compresses large values**. For actual = 500:

Prediction	Error direction	RMSLE contribution	RMSE contribution
100 (under by 400)	Under-predict	$(\log 101 - \log 501)^2 \approx 2.56$	$400^2 = 160,000$
900 (over by 400)	Over-predict	$(\log 901 - \log 501)^2 \approx 0.35$	$400^2 = 160,000$

RMSE is symmetric — same penalty either way. RMSLE penalizes **over-predictions more** than under-predictions for the same absolute error, because log grows slower for large values. Equivalently, RMSLE measures **relative** (proportional) error rather than absolute error, which is appropriate for count data spanning a wide range (1-977).

Q11 — Trade-offs: Simplicity vs Predictive Power

	Simple (Linear Reg)	Complex (GBM)
Interpretability	High	Low
Training speed	Very fast	Slower
Bias	High — misses non-linearity	Low
Variance	Low	Higher — controlled by n_estimators, depth, lr
Overfitting risk	Low	Moderate — needs tuning
Val RMSLE	0.707	0.373

Takeaway: The gain from GBM is large here because the data has strong non-linear, temporal patterns. In a small dataset or when interpretability is critical, regularized linear models are preferable.

Q12 — Why Can't Linear Regression Capture Time-of-Day Effects?

ChromeFileEditViewHistoryBookmarksProfilesTabWindowHelp

argo-rdp.codeargo.net/web-rdp/#/client/aS0wNWE3ZTYyMzkxMTMzYzc0MQBjAGpb24?token=FDC292B4EFE18C036B978FE5DC17C815744843427A6561BAEFAF1A0534D35F18

ML_Assignment1_Soluti...ML - A1 - Thunar

Jupyter ServerHomeML_Assignment1_Soluti...

localhost:8888/notebooks/Downloads/ML - A1/ML_Assignment1_Solution_v2.ipynb?

jupyter ML_Assignment1_Solution_v2 Last Checkpoint: 6 minutes ago

FileEditViewRunKernelSettingsHelp

Not Trusted

Code

JupyterLabPython [conda env:base]

grows slower for large values. Equivalently, RMSLE measures **relative** (proportional) error rather than absolute error, which is appropriate for count data spanning a wide range (1-977).

Q11 — Trade-offs: Simplicity vs Predictive Power

	Simple (Linear Reg)	Complex (GBM)
Interpretability	High	Low
Training speed	Very fast	Slower
Bias	High — misses non-linearity	Low
Variance	Low	Higher — controlled by n_estimators, depth, lr
Overfitting risk	Low	Moderate — needs tuning
Val RMSLE	0.707	0.373

Takeaway: The gain from GBM is large here because the data has strong non-linear, temporal patterns. In a small dataset or when interpretability is critical, regularized linear models are preferable.

Q12 — Why Can't Linear Regression Capture Time-of-Day Effects?

Bike demand by hour is **bimodal** — two distinct peaks (8 AM and 6 PM) with a trough in between. Linear regression fits:

$$\text{count} = \beta_0 + \beta_1 \cdot \text{hour} + \epsilon$$

This is a straight line — it can only represent monotonically increasing or decreasing demand across hours. No single slope captures: low night demand → morning spike → midday dip → evening spike → evening decline.

Solutions used in this assignment:

- hour_sin and hour_cos (cyclic encoding) — captures periodicity
- morning_peak and evening_peak binary flags — directly marks rush hours
- GBM's decision trees — split on hour thresholds naturally, creating step-wise approximations of any shape

This is why our RMSLE dropped from 1.26 (plain linear) to 0.37 (GBM with engineered features).